

ExcursionServices.mjs

This is the business logic for the alert Lambda.

It imports:

```
1 | setAlertRaisedAt
```

from:

```
1 | excursionRepositories.mjs
```

So:

```
1 | Excursion Service
2 |   ↓
3 | setAlertRaisedAt()
4 |   ↓
5 | Excursion Repository
```

Deliberate failure block

You deliberately kept:

```
1 | if (
2 |   message.forceFailure === true
3 | ) {
```

This is **test functionality**.

It intentionally throws:

```
1 | DELIBERATE_ALERT_FAILURE
```

Why?

Because you wanted to prove:

```
1 | Lambda fails
2 |   ↓
3 | SQS retries
4 |   ↓
5 | maximum receives reached
6 |   ↓
7 | message enters DLQ
```

This isn't normal business behaviour.

It's there specifically to demonstrate that your DLQ infrastructure works.

Message validation

The service checks things such as:

```
1 | Do we have readingId?  
2 |  
3 | Do we have branchId?  
4 |  
5 | Is this actually an excursion?
```

If the SNS/SQS message says:

```
1 | status = ok
```

the excursion handler shouldn't silently treat it as an excursion.

Create alertRaisedAt

The service does:

```
1 | const alertRaisedAt =  
2 |   new Date().toISOString();
```

That creates the time the alert was processed.

For example:

```
1 | 2026-09-03T11:15:22.327Z
```

Then it calls:

```
1 | setAlertRaisedAt({  
2 |   readingId,  
3 |   branchId,  
4 |   alertRaisedAt  
5 | });
```

This goes into the repository.